

Project Problem Statement: SECE Intelligent RAG Assistant

1. Project Overview

Issued By: Edsol Innovation

Target Institution: Sri Eshwar College of Engineering

Assigned To: GaneshPrabu BO, Gnan Shruthi

The primary objective of this project is to architect and develop a highly responsive Retrieval-Augmented Generation (RAG) system. This intelligent conversational agent will be designed to comprehensively scrape, comprehend, and retrieve hyper-specific information from the Sri Eshwar College of Engineering official digital presence. The system must go significantly beyond generic FAQs to answer deeply contextual queries (e.g., identifying highly specific student projects, niche technical patterns, and detailed event histories).

2. Core Problem Definition

Currently, the digital infrastructure of modern educational institutions contains vast amounts of unstructured data distributed across hundreds of web pages, academic documents, and event notices. Stakeholders often struggle to find specific, granular information using traditional keyword search. The engineering challenge is to build a robust, automated data pipeline that continuously crawls the college's public domains, intelligently vectorizes the extracted text, and powers an LLM-driven chatbot capable of advanced reasoning over this proprietary dataset.

3. Scope of Work & Technical Phases

Phase 1: Deep Web Scraping & Data Ingestion

- Deploy automated web crawlers to map and extract content from all subdirectories of the college's primary web domain.
- Extract nested text, tabular data, and metadata from HTML nodes, dynamically loaded frontend content, and publicly linked documents (PDFs).
- Implement data cleaning protocols to remove boilerplate navigation, standardize the text corpus, and filter out irrelevant structural code.

Phase 2: Vectorization & RAG Pipeline Integration

- Implement semantic chunking strategies to ensure context is preserved (e.g., keeping project titles inherently linked to the specific student names and departments).
- Generate high-dimensional embeddings for the scraped corpus and persistently store them in a scalable Vector Database.
- Design the retrieval mechanism to prioritize highly relevant chunks, ensuring the language model has the exact context needed for niche queries.

Phase 3: System Architecture & Proposed Tech Stack

To ensure high performance, modularity, and maintainability, the system architecture will leverage a robust full-stack environment integrated with advanced AI endpoints:

System Component	Proposed Technology	Purpose & Integration Strategy
Frontend User Interface	React.js	To build a fluid, interactive conversational UI that manages state efficiently.
Backend API Engine	Node.js & Express.js	Handling asynchronous API requests, session management, and routing to the vector store.
Data & State Storage	MongoDB	Storing chronological chat histories, user analytics, and application state data.
AI & Logic Engine	Google Gemini API / LLM	Processing retrieved contexts, parsing intent, and synthesizing accurate, human-like responses.

4. Strict Acceptance Criteria & Edge Case Handling

- **Deep Contextual Accuracy:** If a user queries a hyper-specific detail such as, *"Has anyone in the college done the Gradient pattern?"*, the RAG system must successfully locate the specific text node within the college's project or event pages and summarize the findings accurately, rather than returning generic programming definitions.
- **Zero Hallucination Tolerance:** If the requested information is absolutely not present within the scraped college data, the bot must transparently state its inability to find the data in the official records rather than generating a plausible but fabricated answer.

- **Continuous Sync & Versioning:** The scraping architecture must support periodic or trigger-based re-indexing to continuously capture newly added announcements, faculty updates, or departmental pages.

5. Expected Deliverables

- A fully functional MERN-stack web application seamlessly integrated with the custom RAG logic.
- A centralized repository containing the automated scraping scripts, data cleaning modules, and database schemas.
- Comprehensive technical documentation outlining the embedding models utilized, the chunking parameters, and instructions for triggering manual knowledge-base updates.